

OrbitFS Architecture & Roadmap

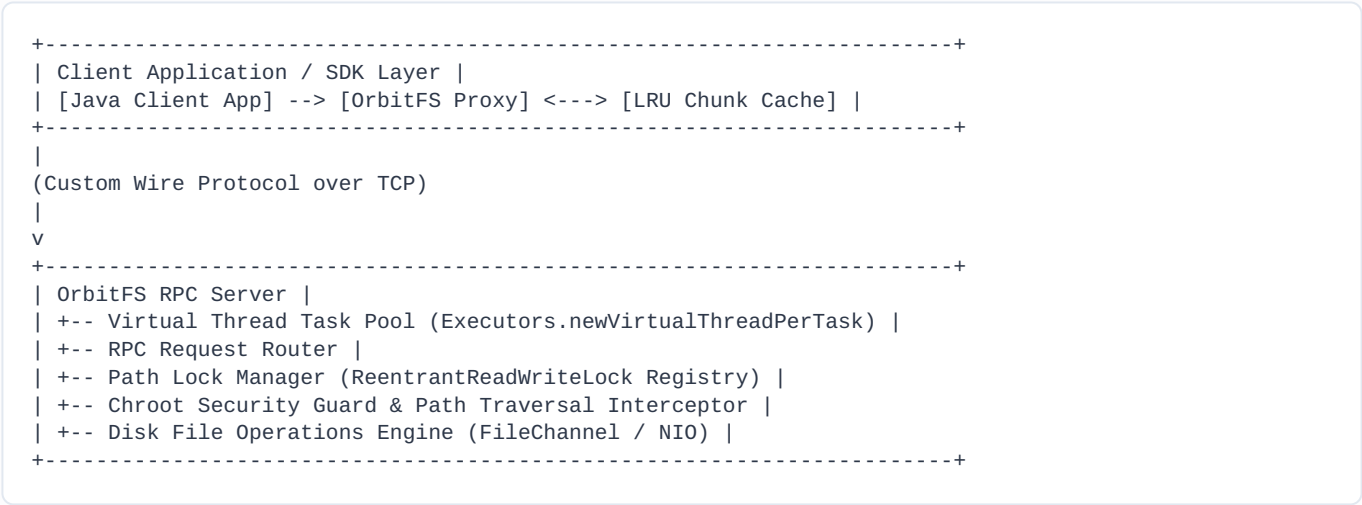
Distributed RPC File System & Security Sandbox Specification

Project Codename:	OrbitFS	Target Stack:	Java 21 LTS (Virtual Threads, NIO)
Architecture:	Distributed RPC / Sandboxed IO	Document Class:	Technical Blueprint & Execution Plan

1. System Overview & Architecture

OrbitFS is a high-throughput, distributed file system engine inspired by **CMU 15-440 Project 1**. Re-engineered in **Java 21**, it replaces POSIX interposition wrappers with a clean, decoupled system architecture:

- **Custom Wire Protocol (RPC):** Direct TCP socket communication using binary length-prefixed headers and JSON frame serialization.
- **Concurrent I/O Engine:** High-throughput server built on Java Virtual Threads (`Executors.newVirtualThreadPerTaskExecutor()`) and `ReentrantReadWriteLock` mechanics for path-level concurrency control.
- **Client-Side Caching Proxy:** LRU chunk cache layer implementing write-back/write-through cache coherency.
- **Security & Sandboxing Guard:** Virtual root (`chroot`) path-resolution engine providing path traversal defenses (`../` neutralization) and canonical boundary enforcement.



2. Protocol Specifications & Frame Format

Communication between the Client Proxy and Server runs over persistent TCP sockets using a **Length-Prefixed Frame Protocol**.

2.1 Wire Frame Structure

Byte Offset	Size	Field Name	Description
0x00 - 0x03	4 Bytes	Magic Header	0x4F 0x52 0x42 0x54 (ORBT)

Byte Offset	Size	Field Name	Description
0x04 - 0x07	4 Bytes	Payload Length	Big-Endian 32-bit Integer (N)
0x08 - N+7	N Bytes	JSON Payload	UTF-8 Encoded RPC Object

2.2 JSON Payload Schemas

RPC Request Schema

```
{
  "requestId": "a1b2c3d4-e5f6-7890-1234-56789abcdef0",
  "method": "FILE_WRITE",
  "path": "/data/logs/app.log",
  "fd": "fd-99812-uuid",
  "offset": 1024,
  "count": 512,
  "dataBase64": "SGVsbG8gV29ybGQgRG1zdHJpYnV0ZWQgRm1sZSBTeXN0ZW0="
}
```

RPC Response Schema

```
{
  "requestId": "a1b2c3d4-e5f6-7890-1234-56789abcdef0",
  "status": "OK",
  "errorCode": 0,
  "bytesProcessed": 512,
  "fd": "fd-99812-uuid",
  "dataBase64": null,
  "stat": {
    "size": 4096,
    "isDir": false,
    "lastModified": 1772323200000
  }
}
```

3. Step-by-Step Implementation Roadmap

Checkpoint 1: RPC Wire Protocol & Custom Transport

Objective: Establish socket connectivity, framing, serialization, and thread dispatching.

- **Define Protocol Objects:** Create `RPCRequest` and `RPCResponse` record/DTO classes.
- **Build Frame Encoders/Decoders:**
 - `FrameEncoder` : Writes `Magic Header (4B) + Length (4B) + Payload (N Bytes)` .
 - `FrameDecoder` : Uses `DataStream.readFully()` to prevent short reads over socket streams.
- **Build Server Listener:** Set up `ServerSocket` listening on a configurable port (e.g., `8080`).
- **Integrate Java 21 Virtual Threads:** Dispatch incoming socket connections using `Executors.newVirtualThreadPerTaskExecutor()` .

Verification Milestone: Execute an integration test where 50 concurrent client threads issue `RPCRequest("PING")` over TCP and receive valid `RPCResponse("PONG")` payloads within 100ms.

Checkpoint 2: Server Storage Engine, File Handles, & Locking

Objective: Implement file system operations (`OPEN` , `READ` , `WRITE` , `CLOSE` , `SEEK` , `STAT`) with strict concurrency protection.

- **File Handle Manager (`FileDescriptorTable`):** Maintain a thread-safe map `ConcurrentHashMap<String, FileChannelHandle>` mapping generated UUID handles to active `FileChannel` streams and offset pointers.
- **Path Lock Manager (`PathLockRegistry`):** Implement a registry mapping canonical file paths to `ReentrantReadWriteLock` instances. Ensure concurrent `READ` calls run in parallel while `WRITE` calls acquire exclusive locks.
- **Implement Core Operations:**
 - `OPEN` : Creates/opens `FileChannel` , registers handle ID, returns handle to client.
 - `WRITE` : Takes handle ID, offset, and payload; applies write at target offset; updates handle position.
 - `READ` : Reads N bytes from specified offset; returns Base64-encoded byte array.
 - `CLOSE` : Flushes unwritten channel data and deregisters file handle ID.

Verification Milestone: Run a concurrency stress test spawning 10 writer threads and 20 reader threads operating on the same target file simultaneously. Verify zero corrupted byte streams or `ConcurrentModificationException` occurrences.

Checkpoint 3: Client Proxy & LRU Caching Layer

Objective: Minimize network roundtrips by introducing a client-side chunk caching strategy.

- **Client Proxy Interface (`OrbitFSClient`):** Implement standard interface abstractions (`open` , `read` , `write` , 未写 `close`).
- **Implement Chunked Cache (`LRUChunkCache`):** Build a block-level cache using Java's `LinkedHashMap` overriding `removeEldestEntry` . Key: `FilePath + ChunkIndex` (Block size = 64 KB).
- **Write-Through / Write-Back Strategy:** On `write()` , update local cache chunk and mark `dirty = true` . On `flush()` / `close()` , send all dirty chunks to server in a single batched RPC write payload.

Verification Milestone: Execute 1,000 sequential 1 KB `read()` operations over a 64 KB file with cache enabled. Verify via server telemetry that only **1 RPC network call** occurred.

Checkpoint 4: Security Sandboxing & Path Guard Engine

Objective: Enforce filesystem boundaries and defend against path traversal exploits.

- **Virtual Root Resolver (`ChrootSandbox Engine`):** Configure a dedicated root workspace (e.g., `/tmp/orbitfs_sandbox`). Resolve paths via `Path.toRealPath()` / `Path.normalize()` .
- **Path Traversal Guard Logic:**

```

Path rootPath = Paths.get("/tmp/orbitfs_sandbox").toRealPath();
Path resolvedPath = rootPath.resolve(requestedPath).normalize();

if (!resolvedPath.startsWith(rootPath)) {
    throw new SecurityException("Access Denied: Path Traversal Detected!");
}

```

- **Operation Whitelisting & Guard Hooks:** Restrict maximum file payload size per single request (Max 10 MB per frame). Block requests targeting hidden files (`.env` , `.git/` , system directories).

Verification Milestone: Run an automated security suite attempting malicious paths (`../../../../etc/passwd` , `/var/run/`). Verify 100% of traversal attempts are rejected with `STATUS_SECURITY_VIOLATION` .

4. Project Directory Structure

```

orbitfs-core/
├── pom.xml
├── src/main/java/org/orbitfs/
│   ├── client/
│   │   ├── OrbitFSClient.java           # Client Proxy Stub Interface
│   │   ├── LRUChunkCache.java           # Client-Side Block Caching Layer
│   │   └── NetworkTransportClient.java   # Socket Connection & Encoding
│   ├── common/
│   │   ├── protocol/
│   │   │   ├── RPCRequest.java          # Request DTO Record
│   │   │   ├── RPCResponse.java         # Response DTO Record
│   │   │   └── FrameCodec.java           # Length-Prefixed Binary Encoder
│   │   └── exception/
│   │       └── SecuritySandboxException.java
│   └── server/
│       ├── OrbitFSServer.java            # Main Listener & Virtual Thread Pool
│       ├── FileDescriptorTable.java       # Thread-Safe FD Handle Registry
│       ├── PathLockRegistry.java          # ReadWriteLock Concurrency Engine
│       ├── StorageEngine.java             # FileChannel Disk IO Implementation
│       └── security/
│           └── ChrootSandboxGuard.java    # Path Traversal & Security Sandbox
└── src/test/java/org/orbitfs/
    ├── ConcurrencyStressTest.java        # Parallel Write/Read Test Suite
    └── PathTraversalSecurityTest.java     # Exploitation Verification Test

```

5. Development Strategy & Generative AI Workflow

Maintain full architectural authority over system design, wire contracts, and concurrency models, while delegating repetitive tasks to Generative AI tools:

1. **Protocol & Spec Generation:** Use LLM prompts to generate `OpenAPI/Swagger` or `JSON Schema` specifications for the RPC payloads.
2. **Boilerplate & Test Suite Generation:** Prompt AI tools to generate JUnit 5 tests for edge-case socket handling, Jackson serialization, and path-resolution unit tests.

3. **Diagrams & Documentation:** Pass domain models and flow specifications to AI to produce detailed Mermaid.js sequence and architecture diagrams for your GitHub `README.md`.

Resume Bullet Blueprint

*"Architected **OrbitFS**, a high-throughput, sandboxed distributed file system in Java 21, featuring a custom JSON-over-TCP binary-framed RPC protocol and Virtual Thread task execution. Implemented **ReentrantReadWriteLock** concurrency controls, a client-side LRU block cache, and a **chroot** path-resolution sandbox defending against path traversal exploits. Leveraged Agentic AI tools for OpenAPI framing, test suite generation, and LLD diagramming while retaining end-to-end system design ownership."*